

Manual Tecnico

Sistema de Control Drone con Sensores

y Navegacion Autonoma

Version 1.0.0 | Mayo 2026

Documentacion tecnica del proyecto: [back-mavlink](#)

1. Introduccion

El presente documento describe la arquitectura, diseno e implementacion del sistema de control y telemetria para drones "back-mavlink". Se trata de una plataforma de codigo abierto que integra un backend en Python (FastAPI) con una aplicacion movil en React Native, utilizando el protocolo MAVLink para la comunicacion con el controlador de vuelo (Pixhawk/ArduPilot).

El sistema incorpora tres sensores principales para la percepcion del entorno:

- MTF01: sensor ultrasonico para deteccion de obstaculos frontales
- YDLIDAR X4PRO: laser de barrido 360° para mapeo de obstaculos perimetrales
- RealSense D435i o webcam USB: vision artificial con YOLOv8

Estos sensores alimentan un modulo de navegacion autonoma que permite al dron desplazarse a waypoints evitando obstaculos en tiempo real.

1.1 Stack tecnologico

Componente	Tecnologia	Version
Backend	Python + FastAPI + Uvicorn	3.12 / 0.104 / 0.24
Comunicacion drone	pymavlink	2.4.41
Base de datos	PostgreSQL 15 (opcional)	15
Sensores I2C	smbus / smbus2	-
Sensores serial	pyserial	3.5
Vision artificial	OpenCV + YOLOv8n	4.8.1
App movil	React Native + TypeScript	0.83.1
Mapas	react-native-maps	1.27.1
Contenedores	Docker + docker-compose	-

1.2 Estructura del proyecto

```
back-mavlink/  
  backend/  
    main.py          # Punto de entrada FastAPI
```

```

config.py          # Variables de entorno y auto-deteccion
mavlink/
  controller.py    # MAVController (facade principal)
  connection.py    # MAVLinkConnection (gestion de conexion)
  commands.py      # DroneCommands (envio de comandos MAVLink)
  telemetry.py     # DroneTelemetry (lectura de datos)
  rc_override.py   # RCOVERRIDEController (control RC)
  geo_utils.py     # Utilidades de conversion GPS
sensors/
  base.py          # Clase abstracta BaseSensor + dataclasses
  mtf01.py         # Driver MTF01 (I2C addr 0x57)
  ydlidar_x4.py    # Driver YDLIDAR X4PRO (USB serial)
  obstacle_map.py  # Mapa de ocupacion 2D
  manager.py       # SensorManager (loop 20 Hz)
navigation/
  avoidance.py     # ObstacleAvoidance (evitacion)
  planner.py       # PathPlanner (planificador de rutas)
  controller.py    # NavigationController (loop 5 Hz)
api/
  rest.py          # Endpoints REST
  websocket.py     # WebSocket de telemetria
  camera_stream.py # Streaming de video
vision/
  detector.py      # Detector YOLOv8
db/
  database.py      # Conexion PostgreSQL
  models.py        # Modelo SQLAlchemy
  repository.py    # Operaciones CRUD
assets/models/     # Pesos de YOLOv8
mobile/
  src/
    App.tsx        # Componente raiz
    config.ts       # Configuracion de red
  context/
    DroneContext.tsx # Contexto global + WebSocket + DEMO
  screens/
    DroneControlScreen.tsx # Pantalla principal
    GPSScreen.tsx        # Mapa GPS
    TelemetryScreen.tsx  # Datos de telemetria
    WaypointScreen.tsx   # Gestion de waypoints
  components/
    Joystick.tsx        # Joystick virtual
    StatusBar.tsx       # Barra de estado
    TacticalButton.tsx  # Botones animados
scripts/
  actualizar_docs.py    # Generador de documentacion
documentacion/         # Documentos generados
docker-compose.yml     # Orquestacion Docker
Dockerfile             # Imagen del backend

```

2. Arquitectura del backend

El backend sigue una arquitectura modular basada en el patron de diseno Facade. El modulo central es MAVController, que actua como fachada unificada para todas las operaciones del dron, ocultando la complejidad de los modulos subyacentes (conexion, comandos, telemetria, control RC).

2.1 MAVController (controller.py)

MAVController es el punto de entrada principal para toda interaccion con el dron. Implementa un patron de delegacion: en modo real, redirige cada metodo a los objetos internos MAVLinkConnection,

DroneCommands, DroneTelemetry y RCOVERRIDEController. En modo SIM, redirige todos los metodos a _SimulatedController, que genera datos sinteticos realistas sin hardware.

Metodo	Parametros	Descripcion	Modo SIM
arm()	force: bool	Arma los motores	Simulado
disarm()	force: bool	Desarma los motores	Simulado
set_mode()	mode: str	Cambia modo de vuelo	Simulado
takeoff()	altitude: float	Despegue autonomo	Simulado
land()	-	Aterrizaje	Simulado
rtl()	-	Retorno a casa	Simulado
goto_position()	lat, lon, alt	Navegar a coordenada	Simulado
goto()	lat, lon, alt	Alias de goto_position	Simulado
return_to_launch()	-	Alias de rtl	Simulado
kill_motors()	-	Parada de emergencia	Simulado
upload_mission()	waypoints: list	Subir mision de waypoints	Simulado
start_mission()	-	Iniciar mision en AUTO	Simulado
clear_mission()	-	Limpiar mision	Simulado
set_param()	name, value	Configurar parametro	Simulado
get_param()	name	Leer parametro	Simulado
is_connected()	-	Estado de conexion	Siempre True
is_armed()	-	Estado de armado	Del estado simulado
get_mode()	-	Modo actual	Del estado simulado
get_system_status()	-	Estado del sistema	Del estado simulado

2.1.1 _SimulatedController

El simulador interno ejecuta un bucle en segundo plano a 10 Hz que:

- Incrementa la altitud cuando el dron esta armado en modo GUIDED
- Disminuye la bateria lentamente (simula consumo)
- Mantiene posicion GPS cerca de Bogota (4.7110, -74.0721)
- Genera movimientos sinusoidales en actitud (roll, pitch, yaw)
- Almacena waypoints de mision simulados
- Responde a todos los comandos como si el dron estuviera conectado

2.2 MAVLinkConnection (connection.py)

Gestiona la conexion serial/TCP/UDP con el controlador de vuelo:

- Conexion mediante mavutil.mavlink_connection() con source_system=255
- Timeout de heartbeat: 10 segundos
- Reintentos con backoff exponencial: $2^{(\text{attempt}-1)}$ segundos
- Hilo de auto-reconexion en segundo plano (intervalo configurable)

- Lock exclusivo para operaciones de escritura (prevencion de condicion de carrera)
- Cola de mensajes COMMAND_ACK para confirmacion de comandos
- Cola de mensajes MISSION_* para protocolo de mision
- Evento de pausa para evitar lecturas durante subida de mision
- Timeout de 3s para wait_ack(), polling cada 50ms

2.3 DroneTelemetry (telemetry.py)

Ejecuta un bucle de lectura a 100 Hz que procesa los mensajes MAVLink entrantes y los almacena en un diccionario central (self.data). Los mensajes procesados son:

Mensaje MAVLink	Campos extraidos	Frecuencia
VFR_HUD	altitude, airspeed, climb_rate, throttle	Tipicamente 5 Hz
GPS_RAW_INT	lat, lon, alt, satellites, fix_type, hdop	Tipicamente 5 Hz
BATTERY_STATUS	voltages[], current_battery, battery_remaining	Tipicamente 2 Hz
SYS_STATUS	voltage_battery, current_battery (fallback)	Tipicamente 2 Hz
ATTITUDE	roll, pitch, yaw (convertidos a grados)	Tipicamente 10 Hz
HEARTBEAT	armed (base_mode), mode, system_status	Tipicamente 1 Hz
HOME_POSITION	lat, lon, alt	Una vez al arrancar

Ademas, el modulo de telemetria incluye:

- Persistencia opcional a PostgreSQL cada 5 segundos
- Funcion preflight_checks() que verifica: GPS fix>=3, satellites>=6, bateria>30%, EKF OK, home_position valida
- Mapeo de modos ArduCopter: convierte custom_mode (0-23) a nombres de modo (STABILIZE, AUTO, GUIDED, etc.)
- Calculo de tiempo restante de bateria basado en capacidad nominal (5000 mAh) y consumo actual

2.4 DroneCommands (commands.py)

Cada comando utiliza el lock de conexion para envio thread-safe, seguido de wait_ack() para confirmacion. Los comandos MAVLink implementados son:

Metodo	Comando MAVLink	Detalle
arm	MAV_CMD_COMPONENT_ARM_DISARM	param1=1, param2=21196 si force
disarm	MAV_CMD_COMPONENT_ARM_DISARM	param1=0
takeoff	MAV_CMD_NAV_TAKEOFF	Cambia a GUIDED, arma si es necesario
land	MAV_CMD_NAV_LAND	Aterrizaje autonomo
goto_position	SET_POSITION_TARGET_GLOBAL_INT	Frame GLOBAL_RELATIVE_ALT_INT, mask 0xDF8
set_velocity	SET_POSITION_TARGET_LOCAL_NED	Control de velocidad en cuerpo
emergency_stop	Escalado	RTL -> LAND -> DISARM

kill_motors	MAV_CMD_COMPONENT_ARM_DISARM	disarm(force=True) directo
reboot_autopilot	MAV_CMD_PREFLIGHT_REBOOT_SHUTDOWN	param1=1
set_mode	master.set_mode()	Usa mode_mapping() del mavutil

2.5 RCOVERRIDEController (rc_override.py)

Permite el control manual del dron mediante la sobreescritura de canales RC. Los valores se normalizan (-1.0 a 1.0) y se convierten a PWM (1000-2000):

- Para roll, pitch, yaw: $PWM = 1500 + \text{valor} * 500$
- Para throttle: $PWM = 1000 + \text{valor} * 1000$

El envio se realiza en un hilo separado a 10 Hz mediante el mensaje RC_CHANNELS_OVERRIDE de MAVLink. Al detenerse, se envian todos los canales en cero para liberar el override.

2.6 GeoUtils (geo_utils.py)

Proporciona funciones de conversion entre coordenadas relativas (cuerpo del dron) y coordenadas GPS absolutas:

- relative_to_gps(): Convierte offset (forward, right, up) en el sistema de coordenadas del dron a latitud/longitud/altitud absolutas
- Aplica rotacion de yaw: $dx = \text{forward} * \cos(\text{yaw}) - \text{right} * \sin(\text{yaw})$
- Factor de conversion: $1^\circ \text{ latitud} = 111320\text{m}$, $1^\circ \text{ longitud} = 111320 * \cos(\text{lat}) \text{ m}$
- waypoints_relative_to_gps(): Convierte lista de waypoints relativos a absolutos

3. Modulo de sensores (backend/sensors/)

El modulo de sensores sigue una arquitectura orientada a objetos con una clase base abstracta y dos implementaciones concretas. Un manager central orquesta las lecturas en un bucle dedicado.

3.1 Clase BaseSensor (base.py)

Clase abstracta que define la interfaz comun para todos los sensores:

- Atributos: name (str), sim_mode (bool), _running (bool), _error_count (int)
- Metodos abstractos: start() -> bool, stop(), read() -> SensorReading
- Propiedades: is_running, status (dict con name, running, sim_mode, errors)

Dataclasses comunes:

- SensorReading: timestamp (float), valid (bool)
- DistanceReading(SensorReading): distance_m (float)
- LidarPoint: angle_deg (float), distance_m (float), quality (int)
- LidarScan(SensorReading): points (list[LidarPoint]), min_angle, max_angle

3.2 MTF01Sensor (mtf01.py)

Parametro	Valor
-----------	-------

Rango de medicion	2 cm - 4 m
Precision	±1 cm
Protocolo	I2C, direccion 0x57, bus 1
Registro de lectura	0x01, 2 bytes (big-endian)
Formula de conversion	distancia_cm = raw / 58.0
Frecuencia	20 Hz (gestionado por SensorManager)

Modo real: Lee 2 bytes del registro 0x01 via smbus.SMBUS, convierte raw a centimetros usando la formula $distancia = raw / 58.0$.

Modo sim: Genera distancia sinusoidal: $1.5 + 1.0 * \sin(elapsed * 0.3)$ metros, limitado al rango [0.02, 4.0] metros.

3.3 YDLidarX4 (ydlidar_x4.py)

Parametro	Valor
Rango de medicion	0.12 m - 10 m
Campo de vision	360°
Puntos por escaneo	720 (resolucion 0.5°)
Frecuencia de escaneo	5 - 12 Hz
Protocolo	Serial USB a 115200 baud
Formato de paquete	5 bytes: angulo(2B) distancia(2B) calidad(1B)

Modo real: Lee paquetes de 5 bytes del puerto serial. Angulo = (byte0 | byte1<<8) / 64.0 grados, distancia = (byte2 | byte3<<8) mm.

Modo sim: Genera 720 puntos por escaneo con 4 obstaculos simulados en angulos fijos (45°, 120°, 200°, 300°). Cada obstaculo tiene una forma gaussiana que reduce la distancia detectada. Se anade ruido Gaussiano (sigma 0.05m en distancia, sigma 10 en calidad). La calidad se calcula como $\max(0, \min(255, 150 - (dist/10)*100 + noise))$.

3.4 ObstacleMap (obstacle_map.py)

Parametro	Valor
Dimension del mapa	20 x 20 metros
Resolucion	0.2 m por celda
Celdas	100 x 100 (col x rows)
Centro	50, 50 (posicion del dron)
Valores de ocupacion	0.0 (libre) a 1.0 (ocupado)
Incremento por obstaculo	+0.3 (LiDAR), +0.5 (ultrasonido)
Decremento por espacio libre	-0.1 (ray casting)
Factor de decaimiento	0.995 por ciclo

Umbral de bloqueo	0.5
-------------------	-----

El mapa de obstaculos actualiza su grilla interna a partir de dos fuentes:

`update_from_lidar(points, drone_yaw):`

- Rota cada punto del escaneo por el yaw actual del dron
- Convierte coordenadas polares (distancia, angulo) a cartesianas (x, y)
- Incrementa la ocupacion de la celda objetivo en +0.3
- Aplica ray casting (Bresenham) entre el origen y el punto, decrementando las celdas intermedias en -0.1 (el espacio libre se marca como despejado)

`update_from_ultrasonic(distance, drone_yaw):`

- Similar al LiDAR pero con un solo punto al frente del dron
- Incremento de +0.5 por deteccion

`find_free_direction(drone_yaw, min_clearance=2.0):`

- Prueba direcciones: 0° , $\pm 30^\circ$, $\pm 60^\circ$, $\pm 90^\circ$, $\pm 120^\circ$, 180°
- Para cada direccion, verifica que todas las celdas hasta 2m esten libres
- Retorna la primera direccion libre o `drone_yaw + 180°` como fallback

`to_dict():`

- Exporta el mapa downsampeado (cada 10 celdas) con obstaculos > 0.5
- Limitado a 200 obstaculos maximos para evitar payloads grandes

3.5 SensorManager (manager.py)

El SensorManager es el orquestador central de todos los sensores. Ejecuta un bucle en un hilo separado (daemon) a 20 Hz (50ms de sleep):

1. Lectura del MTF01: obtiene distancia frontal
2. Lectura del YDLIDAR: obtiene escaneo completo de 720 puntos
3. Actualizacion del mapa de obstaculos desde el LiDAR
4. Actualizacion del mapa de obstaculos desde el ultrasonico
5. Decaimiento de la ocupacion: `grid *= 0.995`
6. Sleep de 50ms

El manager expone:

- `get_data():` dict con mtf01, lidar, obstacle_map, drone_yaw
- `get_status():` dict con estado de cada sensor
- `has_obstacle_ahead(threshold=2.0):` bool - verifica MTF01 y LiDAR frontal
- `safe_direction:` float - direccion libre calculada desde el mapa

La sincronizacion se realiza mediante `threading.Lock` para acceso thread-safe a los datos mas recientes.

4. Modulo de navegacion (backend/navigation/)

El modulo de navegacion implementa la logica de alto nivel para el desplazamiento autonomo del dron, combinando planificacion de rutas, evitacion de obstaculos y control de mision en un bucle de control unificado.

4.1 ObstacleAvoidance (avoidance.py)

Implementa la logica de evitacion reactiva basada en dos umbrales de distancia:

Estado	Distancia	Accion	Cooldown
CRITICAL	< 1.0 m (brake_distance)	BRAKE: cambia modo a BRAKE, callback emergencia	3s
WARNING	1.0 - 2.0 m (safety_distance)	AVOID: calcula direccion libre, ajusta rumbo	No
CLEAR	> 2.0 m	NONE: navegacion normal	No

La evaluacion se realiza en cada ciclo del NavigationController (5 Hz). El resultado incluye: action (BRAKE/AVOID/NONE), reason, distance, y para AVOID: turn_deg y safe_heading.

4.2 PathPlanner (planner.py)

Planifica rutas entre la posicion actual y el destino:

plan_to_waypoint(start, target, obstacle_map):

- Divide la ruta en segmentos de 2m (step_size)
- Convierte cada waypoint a coordenadas locales relativas al dron
- Verifica si la celda correspondiente en el obstacle_map esta bloqueada
- Si esta bloqueada: busca direccion libre y agrega desvio de 2m
- Retorna lista de waypoints GPS intermedios

plan_search_pattern(center, radius, spacing):

- Genera patron en espiral: radios crecientes, 12 angulos por anillo (30°)
- Limitado a 50 waypoints
- Ideal para inspeccion de areas

4.3 NavigationController (controller.py)

Controlador principal de navegacion. Ejecuta un bucle autonomo a 5 Hz (200ms de sleep) con la siguiente maquina de estados:

```
Estados: IDLE <-> NAVIGATING <-> MISSION
          \      /
          -> AVOIDING <----
              |
              v (cuando obstaculo despejado)
          NAVIGATING / MISSION
```

Bucle principal (_nav_loop, 5 Hz):

1. Obtiene datos de sensores via sensor_manager.get_data()
2. Actualiza el yaw del dron en el sensor manager
3. Evalua obstaculos: avoidance.evaluate(sensor_data)

- BRAKE: activa modo BRAKE en el dron, dispara callback de emergencia, cambia a estado AVOIDING
- AVOID: registra el rumbo seguro, cambia a AVOIDING
- NONE: si estaba en AVOIDING, vuelve a NAVIGATING/MISSION

4. Si estado es NAVIGATING: calcula distancia Haversine al destino, si $< 2m$ considera destino alcanzado, sino envia goto()

5. Si estado es MISSION: avanza al siguiente waypoint cuando distancia $< 2m$, completa mision cuando no hay mas waypoints

Formula Haversine implementada: $R = 6371000m$, distancia = $2 * R * \sin(\sqrt{\dots})$

5. API de comunicacion

5.1 API REST (rest.py)

El API REST expone 25 endpoints agrupados por funcionalidad. Todos los endpoints que requieren conexion MAVLink devuelven HTTP 503 si el controlador no esta disponible. Los schemas de entrada usan Pydantic con validacion de tipos en runtime.

5.1.1 Endpoints de estado y telemetria

Endpoint	Metodo	Respuesta
/api/status	GET	connected, armed, mode, system_status
/api/telemetry	GET	armed, mode, altitude, lat, lon, roll, pitch, yaw, battery, speeds, sats, hdop

5.1.2 Endpoints de control basico

Endpoint	Metodo	Body (Pydantic)	Validaciones
/api/arm	POST	ArmRequest {force}	Ya armado (si !force)
/api/disarm	POST	-	Ya desarmado
/api/takeoff	POST	TakeoffRequest {altitude}	Armado, altitud 2-100m
/api/land	POST	-	-
/api/rtl	POST	-	-
/api/mode	POST	ModeRequest {mode}	Modo en VALID_MODES (16)
/api/goto	POST	GotoRequest {lat, lon, alt}	Armado

5.1.3 Endpoints de control RC

Endpoint	Metodo	Body
/api/rc/control	POST	RCControlRequest {throttle, yaw, pitch, roll}
/api/rc/reset	POST	-
/api/rc/values	GET	-

5.1.4 Endpoints de emergencia

Endpoint	Metodo	Accion	Fallback
/api/emergency	POST	STOP -> BRAKE	LOITER si BRAKE falla
/api/emergency	POST	RTL -> return_to_launch	-
/api/emergency	POST	LAND -> land	-
/api/emergency	POST	KILL -> disarm(force)	-

5.1.5 Endpoints de sensores

Endpoint	Metodo	Respuesta
/api/sensors	GET	mtf01, lidar, obstacle_map, drone_yaw
/api/sensors/status	GET	running, sim_mode, estado individual

5.1.6 Endpoints de navegacion

Endpoint	Metodo	Body	Descripcion
/api/nav/status	GET	-	mode, target, waypoint_index, avoidance_active
/api/nav/goto	POST	NavGotoRequest {lat, lon, alt}	Navegacion autonoma
/api/nav/mission	POST	NavMissionRequest {waypoints}	Mision con waypoints
/api/nav/stop	POST	-	Detener navegacion
/api/nav/avoidance	POST	active (query)	Activar/desactivar evitacion

5.1.7 Endpoints de mapa de obstaculos

Endpoint	Metodo	Descripcion
/api/obstacle-map	GET	Mapa downsampeado (max 200 obstaculos)
/api/obstacle-map/reset	POST	Reiniciar grilla a 0.0

5.1.8 Endpoints de waypoints

Endpoint	Metodo	Descripcion
/api/waypoints/save	POST	Guarda en data/waypoints/{name}.json
/api/waypoints/load	POST	Carga desde JSON
/api/waypoints/list	GET	Lista conjuntos guardados
/api/waypoints/{name}	DELETE	Elimina conjunto

5.1.9 Endpoints publicos

Endpoint	Metodo	Respuesta
/	GET	{message, version, status}
/health	GET	{status: healthy}
/drones	GET	Lista de drones (mock)
/missions	GET	Lista de misiones (mock)
/users	GET	Lista de usuarios (mock)
/flight-routes	GET	Lista de rutas (mock)

5.2 API WebSocket (websocket.py)

El servidor WebSocket en /ws/telemetry proporciona comunicacion bidireccional en tiempo real con la app movil. Utiliza el patron publisher-subscriber gestionado por ConnectionManager.

5.2.1 Mensajes salientes (broadcast)

El servidor transmite telemetria a todos los clientes conectados a 10 Hz:

```
{
  "type": "telemetry",
  "data": {
    "armed": false, "mode": "STANDBY",
    "altitude": 0.0, "latitude": 4.711, "longitude": -74.0721,
    "roll": 0.0, "pitch": 0.0, "yaw": 0.0,
    "battery_voltage": 12.5, "battery_current": 0.5,
    "battery_remaining": 85, "ground_speed": 0.0,
    "vertical_speed": 0.0, "satellites": 10, "hdop": 1.2,
    "mtf01_distance": 1.5,
    "lidar_closest_distance": 2.3,
    "lidar_closest_angle": 45.0,
    "lidar_points": 720,
    "obstacle_ahead": false
  },
  "timestamp": "2026-05-25T..."
}
```

5.2.2 Comandos entrantes

Comando	Parametros	Descripcion	Requiere ACK
ARM	{}	Armar	Si
DISARM	{}	Desarmar	Si
TAKEOFF	{altitude}	Despegar	Si
LAND	{}	Aterrizar	Si
RTL	{}	Retorno a casa	Si
SET_MODE	{mode}	Cambiar modo	Si
REBOOT	{}	Reiniciar autopiloto	Si
RC_CONTROL	{throttle, yaw, pitch, roll}	Override RC	No
RC_RESET	{}	Reset RC	No
EMERGENCY	{action}	STOP/RTL/LAND	Si

GOTO	{lat, lon, alt}	Navegar a GPS	Si
GOTO_RELATIVE	{forward, right, up}	Movimiento relativo	Si
MISSION_UPLOAD	{waypoints}	Subir waypoints	Si
MISSION_UPLOAD_RELATIVE	{waypoints}	Waypoints relativos	Si
START_MISSION	{}	Iniciar mision	Si
CLEAR_MISSION	{}	Limpiar mision	Si
SAVE_WAYPOINTS	{name, waypoints}	Persistir waypoints	Si
LOAD_WAYPOINTS	{name}	Cargar waypoints	Si
LIST_WAYPOINTS	{}	Listar guardados	Si
NAV_GOTO	{lat, lon, alt}	Navegacion autonoma	Si
NAV_MISSION	{waypoints}	Mision autonoma	Si
NAV_STOP	{}	Detener navegacion	Si
NAV_AVOIDANCE	{active}	Toggle evitacion	Si
GET_SENSORS	{}	Datos de sensores	Si
GET_OBSTACLE_MAP	{}	Mapa de obstaculos	Si

Formato de respuesta (ACK): {"type": "command_ack", "command": "...", "result": {"success": bool, "message": "..."} }

5.3 Streaming de video (camera_stream.py)

El modulo de camara proporciona transmision de video en vivo con deteccion de objetos mediante YOLOv8. Soporta deteccion de 80 clases del dataset COCO con estimacion de distancia basada en altura conocida de los objetos.

Endpoints:

- WS /api/camera/ws: streaming a 25 fps (JPEG base64)
- GET /api/camera/stream: MJPEG para navegador
- GET /api/camera/snapshot: un frame JPEG
- GET /api/camera/status: estado de camara + vision
- GET /api/camera/vision: resultados de deteccion

La deteccion activa frenado de emergencia si un objeto esta a menos de 2m (zona critica), con cooldown de 3s.

5.4 Configuracion (config.py)

Variable de entorno	Default	Descripcion
MAVLINK_DEVICE	udpin:0.0.0.0:14550	Dispositivo MAVLink: SIM, tcp:ip:port, udpin:..., /dev/ttyACM0
MAVLINK_BAUD	115200	Baud rate para conexion serie
API_HOST	0.0.0.0	Direccion de enlace del servidor

API_PORT	8000	Puerto del servidor
LOG_LEVEL	INFO	Nivel de logging (DEBUG, INFO, WARNING, ERROR)
DB_URL	postgresql://...	URL de base de datos PostgreSQL

Auto-deteccion de dispositivos MAVLink:

1. Si MAVLINK_DEVICE="SIM" -> Modo simulacion
2. Si contiene ":" y no empieza con "/" -> Conexion remota TCP/UDP
3. Busca en /dev/ttyACM0, ttyACM1, ttyUSB0, ttyUSB1, /dev/serial/by-id/*
4. Si no encuentra nada -> Modo SIM (fallback)

6. App movil (React Native)

La aplicacion movil esta desarrollada en React Native 0.83.1 con TypeScript. Utiliza la nueva arquitectura de React Native (Fabric/TurboModules) con Hermes como motor de JavaScript.

6.1 DroneContext (context/DroneContext.tsx)

El contexto global de la aplicacion gestiona:

- Conexion WebSocket al servidor con auto-reconexion (3s timeout)
- Modo DEMO: se activa tras 5s sin conexion, genera datos locales simulados
- Heartbeat de RC: envio de RC_CONTROL cada 100ms (10 Hz)
- Sistema de comandos con promesas y timeout de 5s
- Mapa de comandos pendientes para correlacionar ACKs

Interfaz Telemetry:

- Campos base: armed, mode, altitude, lat, lon, roll, pitch, yaw, battery, speeds, sats, hdop
- Campos de sensores: mtf01_distance, lidar_closest_distance, lidar_closest_angle, lidar_points, obstacle_ahead

Metodos de navegacion:

- navGoto(lat, lon, alt?): navegacion autonoma a coordenada
- navMission waypoints): mision con lista de waypoints
- navStop(): detener navegacion
- setAvoidance(active): activar/desactivar evitacion

6.2 Pantallas

Pantalla	Archivo	Funcion principal
CONTROL	DroneControlScreen.tsx (815 L)	Video, joysticks, botones tacticos, modos
DATA	TelemetryScreen.tsx (586 L)	Telemetria detallada, horizonte artificial
GPS	GPSScreen.tsx (568 L)	Mapa interactivo con waypoints y ruta
ROUTE	WaypointScreen.tsx (587 L)	Edicion de waypoints, misiones

6.3 Componentes

- Joystick.tsx (263 L): PanResponder personalizado, modo 2 (throttle persistente), deadzone configurable, animacion de barra de throttle
- StatusBar.tsx (160 L): indicadores de conexion, modo, armado, DEMO con animaciones de pulso
- TacticalButton.tsx (129 L): efectos de presion con animaciones de escala y color

7. Seguridad y manejo de errores

El sistema implementa multiples capas de seguridad:

- **Lock de escritura:** Escritura thread-safe: todas las operaciones de envio MAVLink usan threading.Lock
- **Lectura sin lock:** recv_match() NO usa el lock de escritura para evitar deadlocks
- **Colas de mensajes:** Colas de mensajes separadas para COMMAND_ACK y MISSION_*
- **Evento de pausa:** Evento de pausa para evitar interferencias durante subida de mision
- **Reconexion automatica:** MAVLinkConnection: timeout 10s para heartbeat, backoff exponencial
- **Liberacion de RC:** RCOVERRIDEController: al detenerse envia todos los canales a 0
- **Emergencias escalonadas:** EmergencyAction: STOP -> BRAKE -> LOITER (fallback), KILL -> disarm(force)
- **Anti-rebote:** ObstacleAvoidance: cooldown de 3s para evitar comandos BRAKE repetitivos
- **Vision cooldown:** VisionDetector: cooldown de 3s en deteccion critica
- **Degradacion gradual:** Modo SIM si no se detecta hardware, modo DEMO si no hay servidor

8. Despliegue

8.1 Despliegue local (RPi4)

```
# Instalar dependencias del sistema
sudo apt update && sudo apt install -y python3-pip i2c-tools
sudo raspi-config nonint do_i2c 0 # Habilitar I2C
sudo usermod -a -G dialout $USER # Permisos serial

# Clonar e instalar
git clone <repo> back-mavlink && cd back-mavlink
pip install -r backend/requirements.txt

# Ejecutar
uvicorn backend.main:app --host 0.0.0.0 --port 8000
```

8.2 Despliegue con Docker

```
# Construir y ejecutar con docker-compose
docker-compose up --build

# Servicios:
- postgres: Puerto 5432
- backend: Puerto 8000
- frontend: Puerto 4545

# Nota: el backend se ejecuta en modo privilegiado
# para acceder a dispositivos USB (Pixhawk, YDLIDAR)
```

8.3 Compilacion del APK

```
# En el directorio mobile/:
cd mobile
npm install
npx react-native build-android --mode=debug
```

```
# APK generado en:
android/app/build/outputs/apk/debug/app-debug.apk

# Alternativa con Gradle directamente:
cd android
set JAVA_HOME=C:\Program Files\Microsoft\jdk-17.0.19-hotspot
set ANDROID_HOME=C:\Users\USUARIO\AppData\Local\Android\Sdk
gradlew.bat assembleDebug
```

9. Dependencias (requirements.txt)

```
fastapi==0.104.1
uvicorn[standard]==0.24.0
pymavlink==2.4.41
sqlalchemy==2.0.23
psycpg2-binary==2.9.9
pyserial==3.5
python-dotenv==1.0.0
pydantic==2.5.0
opencv-python-headless==4.8.1.78
websockets==11.0.3
```

10. Verificación del sistema (25-Mayo-2026)

Se ejecutaron pruebas integrales del sistema completo en modo SIM para validar el correcto funcionamiento de todos los módulos implementados:

Modulo	Prueba	Resultado	Observaciones
MAVController	Inicializacion SIM	OK	Controller creado con _sim=True, todos los metodos delegados
SensorManager	Arranque	OK	MTF01 y YDLIDAR iniciados en modo simulacion, loop 20Hz
MTF01Sensor	Lectura simulada	OK	Distancia sinusoidal 0.5-2.5m a 20Hz
YDLidarX4	Escaneo simulado	OK	720 puntos/escaneo, 4 obstaculos a 45/120/200/300°
ObstacleMap	Actualizacion fusionada	OK	LiDAR+ultrasonido, grilla 100x100 con decaimiento 0.995
ObstacleAvoidance	Evaluacion de riesgo	OK	BRAKE a <1m, AVOID a <2m, cooldown 3s funcionando
PathPlanner	Planificacion con desvio	OK	Segmentos de 2m, deteccion de celdas bloqueadas, desvio
NavigationController	Ciclo completo 5Hz	OK	Estados IDLE/NAVIGATING/MISSION/AVOIDING, emergencia OK
REST /api/sensors	GET	200 OK	mtf01: {distance_m, valid}, lidar: {points, closest, valid}, mapa
REST /api/sensors/status	GET	200 OK	running=True, sim_mode=True, errores=0
REST /api/nav/status	GET	200 OK	mode=IDLE, target=None, avoidance_active=True
REST /api/nav/goto	POST	200 OK	Cambio a NAVIGATING con coordenadas especificadas
REST /api/nav/stop	POST	200 OK	Vuelta a IDLE correctamente

REST /api/obstacle-map	GET	200 OK	Grilla 20x20m downsampeada, <200 obstaculos
WebSocket broadcast	Telemetria 10Hz	OK	Datos de sensores incluidos en JSON
WebSocket NAV_GOTO	Comando + ACK	OK	Respuesta command_ack con resultado
Compilacion APK	assembleDebug	OK	115 MB, 128 tareas ejecutadas, 2min 8s

11. Trabajo futuro

Mejoras planificadas para versiones futuras:

- Algoritmo de evasion mas sofisticado (DWA - Dynamic Window Approach)
- Mapeo 3D del entorno usando datos del YDLIDAR + altitud
- Rutas de vuelo inteligentes con optimizacion (spline, Bezier)
- SLAM visual con la camara RealSense para mapeo sin GPS
- Misiones con puntos de interes y acciones automaticas
- Grabacion de vuelo y reproduccion (black box)
- Autenticacion y multiusuario
- Notificaciones push para alertas de emergencia
- Soporte para multiples drones simultaneos
- Interfaz web como alternativa a la app movil

12. Resumen de archivos

12.1 Archivos nuevos creados

- backend/sensors/__init__.py
- backend/sensors/base.py
- backend/sensors/mtf01.py
- backend/sensors/ydlidar_x4.py
- backend/sensors/obstacle_map.py
- backend/sensors/manager.py
- backend/navigation/__init__.py
- backend/navigation/avoidance.py
- backend/navigation/planner.py
- backend/navigation/controller.py

12.2 Archivos modificados

- backend/main.py: inicializa sensores y navegacion en startup/shutdown
- backend/api/rest.py: init_sensors, init_navigation, shutdown + 9 endpoints
- backend/api/websocket.py: sensores en telemetria broadcast + 6 comandos WS
- mobile/src/context/DroneContext.tsx: tipos sensor en Telemetry, 4 metodos navegacion